Academic Algorithm Explanations & Pseudocode Reference

Smart Multipurpose Emergency Drone Delivery and Route Optimization System *Final-Year B.Tech CSE Academic Submission Reference*

---

1. Overview

This document provides detailed algorithmic explanations, mathematical formulations, and formal pseudocode for the nine core algorithms implemented in the system: 1. **Candidate Route Generation** 2. **Shortest-Feasible Route Selection** 3. **Weighted Multi-Objective Route Optimization** 4. **Battery Energy Consumption & Reserve Calculation** 5. **Payload Feasibility Validation** 6. **Priority Best-Fit Multi-Drone Allocation** 7. **First-Come-First-Served (FCFS) Baseline Allocation** 8. **Dynamic In-Flight Rerouting Engine** 9. **Advisory Ridge Regression Prediction**

---

2. Algorithm 1: Candidate Route Generation

2.1 Concept & Intuition

Unlike road vehicles that follow predefined asphalt street networks, aerial multi-rotors fly in 3D Very Low Level (VLL) airspace. To provide choices between shortest distance, lateral hazard circumvention, and vertical obstacle clearance, the generator deterministically constructs **four distinct 3D flight corridors**: 1. **Direct Air Corridor (`CORRIDOR_DIRECT`):** The direct geodesic line at nominal cruise altitude ($60\ \text{m}$ AGL). 2. **Northern Peripheral Corridor (`CORRIDOR_NORTH_DETOUR`):** Waypoints deflected northward by $+1.2\ \text{km}$ at the flight path midpoint to navigate around central obstacles or urban turbulence. 3. **Southern Peripheral Corridor (`CORRIDOR_SOUTH_DETOUR`):** Waypoints deflected southward by $-1.2\ \text{km}$ at the flight path midpoint. 4. **High-Altitude Safety Corridor (`CORRIDOR_HIGH_ALTITUDE`):** Elevated to $120\ \text{m}$ AGL to clear tall buildings and high trees, at the cost of vertical climb energy.

2.2 Mathematical Formulation

Given origin coordinates (ϕ_1, λ_1) and destination coordinates (ϕ_2, λ_2): - Midpoint coordinates:

$$\phi_m = \frac{\phi_1 + \phi_2}{2}, \quad \lambda_m = \frac{\lambda_1 + \lambda_2}{2}$$

- North/South lateral offset $\Delta_{\text{lat}} = \frac{\delta}{111.0\ \text{km/deg}}$ where $\delta = 1.2\ \text{km}$:

$$\phi_{\text{north}} = \phi_m + \Delta_{\text{lat}}, \quad \phi_{\text{south}} = \phi_m - \Delta_{\text{lat}}$$

2.3 Formal Pseudocode

```
Algorithm: Generate3DCandidateCorridors
Input: pickup (lat1, lon1), dest (lat2, lon2), cruise_speed_kmh
Output: List of RouteCandidate objects

1: candidates = []
2: direct_distance = Haversine(lat1, lon1, lat2, lon2)
3: mid_lat = (lat1 + lat2) / 2.0
4: mid_lon = (lon1 + lon2) / 2.0
5: lat_offset_1_2km = 1.2 / 111.0  // approx 0.0108 degrees

// Corridor 1: Direct Air Corridor
6: c_direct = CreateCandidate("CORRIDOR_DIRECT", waypoints=[(lat1, lon1, 60m), (lat2, lon2, 60m)]
7: c_direct.distance = direct_distance
8: c_direct.penalty = 0.0
9: candidates.append(c_direct)

// Corridor 2: Northern Detour
10: c_north = CreateCandidate("CORRIDOR_NORTH_DETOUR", waypoints=[(lat1, lon1, 60m), (mid_lat + la
11: c_north.distance = Haversine(lat1, lon1, mid_lat + lat_offset_1_2km, mid_lon) + Haversine(mid_
12: c_north.penalty = (c_north.distance / direct_distance - 1.0) * 1.5
13: candidates.append(c_north)

// Corridor 3: Southern Detour
14: c_south = CreateCandidate("CORRIDOR_SOUTH_DETOUR", waypoints=[(lat1, lon1, 60m), (mid_lat - la
15: c_south.distance = Haversine(lat1, lon1, mid_lat - lat_offset_1_2km, mid_lon) + Haversine(mid_
16: c_south.penalty = (c_south.distance / direct_distance - 1.0) * 1.5
17: candidates.append(c_south)

// Corridor 4: High Altitude Safety Corridor
18: c_high = CreateCandidate("CORRIDOR_HIGH_ALTITUDE", waypoints=[(lat1, lon1, 120m), (lat2, lon2_
19: c_high.distance = direct_distance
20: c_high.penalty = (120m - 60m) * 0.01  // 0.60 climb penalty
21: candidates.append(c_high)

22: Return candidates
```

---

3. Algorithm 2: Shortest-Feasible Route Selection

3.1 Concept & Intuition

The shortest-feasible algorithm serves as the primary **baseline comparator**. It evaluates all generated corridors and selects the route with the minimum ground track distance that satisfies physical feasibility constraints. It completely ignores energy drain, climb penalties, and wind shear.

3.2 Formal Pseudocode

```
Algorithm: SelectShortestFeasibleRoute
Input: candidates (List of RouteCandidate), feasibility_checker
Output: selected_candidate (RouteCandidate)

1: feasible_list = []
2: for candidate in candidates do:
3:     if feasibility_checker.is_feasible(candidate) == True then:
4:         feasible_list.append(candidate)
5:     end if
6: end for

7: if feasible_list is EMPTY then:
8:     raise NoFeasibleRouteError("All candidate corridors violate physical or weather constraints
9: end if

10: shortest_candidate = feasible_list[0]
11: for candidate in feasible_list do:
12:     if candidate.distance_km < shortest_candidate.distance_km then:
13:         shortest_candidate = candidate
14:     else if candidate.distance_km == shortest_candidate.distance_km and candidate.id < shortes
15:         shortest_candidate = candidate  // Deterministic tie-breaker
16:     end if
17: end for

18: Return shortest_candidate
```

---

4. Algorithm 3: Weighted Multi-Objective Route Optimization

4.1 Concept & Intuition

Selecting routes based purely on distance can lead to dangerous decisions—for instance, choosing a high-altitude route with identical 2D distance that burns excessive battery power climbing vertically. The **Weighted Multi-Objective Route Optimizer** uses Min-Max Normalized Scalarization across four competing objectives: 1. Distance (d) — Weight: 0.25 2. Duration (t) — Weight: 0.35 3. Modeled Energy Consumption (e) — Weight: 0.30 4. Structural Maneuver & Climb Penalty (p) — Weight: 0.10

4.2 Mathematical Expression

For each feasible candidate $c \in \mathcal{C}_{\text{feasible}}$:

$$x^* = \frac{x - x_{\min}}{x_{\max} - x_{\min} + 10^{-9}}$$

$$J(c) = 0.25 \cdot d_c^* + 0.35 \cdot t_c^* + 0.30 \cdot e_c^* + 0.10 \cdot p_c^*$$

$$c^* = \arg\min_{c \in \mathcal{C}_{\text{feasible}}} J(c)$$

4.3 Formal Pseudocode

```
Algorithm: WeightedMultiObjectiveOptimization
Input: candidates (List of RouteCandidate), feasibility_checker, weights (wd=0.25, wt=0.35, we=0.
Output: optimal_candidate (RouteCandidate)

1: feasible_candidates = [c for c in candidates if feasibility_checker.is_feasible(c)]
2: if feasible_candidates is EMPTY then:
3:     raise NoFeasibleRouteError("No candidate corridors meet safety thresholds.")
4: end if

// Extract metric bounds across all candidates for normalization
5: min_d = min(c.distance_km for c in candidates); max_d = max(c.distance_km for c in candidates)
6: min_t = min(c.duration_min for c in candidates); max_t = max(c.duration_min for c in candidate
7: min_e = min(c.energy_wh for c in candidates);    max_e = max(c.energy_wh for c in candidates)
8: min_p = min(c.penalty for c in candidates);      max_p = max(c.penalty for c in candidates)

9: for c in feasible_candidates do:
10:    norm_d = (c.distance_km - min_d) / (max_d - min_d + 1e-9)
11:    norm_t = (c.duration_min - min_t) / (max_t - min_t + 1e-9)
12:    norm_e = (c.energy_wh - min_e) / (max_e - min_e + 1e-9)
13:    norm_p = (c.penalty - min_p) / (max_p - min_p + 1e-9)
14:    c.objective_score = (wd * norm_d) + (wt * norm_t) + (we * norm_e) + (wp * norm_p)
15: end for

16: optimal_candidate = Sort(feasible_candidates, by=[c.objective_score ASC, c.id ASC])[0]
17: Return optimal_candidate
```

---

5. Algorithm 4: Battery Energy & Reserve Calculation

5.1 Concept & Intuition

Multi-rotor drones consume electric power to stay airborne (induced hovering power) and to overcome aerodynamic drag during forward cruise. To prevent fatal mid-air battery exhaustion, aviation guidelines mandate a **non-depletable battery reserve margin of at least 15%**.

5.2 Mathematical Formulation

- Base Drone Power: $P_{\text{base}} = 150.0 \text{ Watts}$
- Payload Power Multiplier: $P_{\text{cargo}} = 30.0 \text{ Watts per kg} \times m_{\text{cargo}}$
- Headwind Power Multiplier: $P_{\text{wind}} = 4.5 \text{ Watts per } (\text{m/s}) \times v_{\text{wind}}$
- Climb Power Multiplier: $P_{\text{climb}} = 0.25 \text{ Watts per meter altitude difference}$

$$\text{Total Power } P_{\text{total}} = P_{\text{base}} + P_{\text{cargo}} + P_{\text{wind}} + P_{\text{climb}}$$

$$\text{Estimated Energy Drain (Watt-Hours)} = P_{\text{total}} \times \left(\frac{t_{\text{duration_minutes}}}{60.0} \right)$$

$$\text{Usable Battery Capacity} = E_{\text{current_battery}} \times (1.0 - 0.15)$$

$$\text{Safety Constraint: } E_{\text{required}} \leq \text{Usable Battery Capacity}$$

5.3 Formal Pseudocode

```
Algorithm: EvaluateBatteryFeasibility
Input: current_battery_wh, battery_capacity_wh, required_energy_wh, reserve_margin=0.15
Output: is_battery_sufficient (bool), explanation (string)

1: usable_battery_wh = current_battery_wh * (1.0 - reserve_margin)
2: if required_energy_wh > usable_battery_wh then:
3:     shortfall_wh = required_energy_wh - usable_battery_wh
4:     explanation = "Insufficient battery: route requires " + Round(required_energy_wh, 2) + " Wh " +
                     "usable battery is " + Round(usable_battery_wh, 2) + " Wh " +
                     "(current: " + Round(current_battery_wh, 2) + " Wh with 15% mandatory reserve
5:     Return False, explanation
6: else:
7:     remaining_after_flight_wh = current_battery_wh - required_energy_wh
8:     Return True, "Battery check passed. Estimated arrival battery: " + Round(remaining_after_f
9: end if
```

---

6. Algorithm 5: Payload Feasibility Validation

6.1 Concept & Intuition

Exceeding the maximum structural payload capacity causes multi-rotor motors to overheat, destabilizes PID flight controllers, and may lead to catastrophic loss of control. The payload validator verifies cargo mass before any corridor calculations occur.

6.2 Formal Pseudocode

```
Algorithm: ValidatePayloadFeasibility
Input: cargo_weight_kg, drone_max_payload_kg
Output: is_valid (bool), rejection_reason (string)

1: if cargo_weight_kg <= 0.0 then:
2:     Return False, "Invalid cargo mass: payload must be greater than 0 kg."
3: end if

4: if cargo_weight_kg > drone_max_payload_kg then:
5:     rejection_reason = "Payload exceeds capacity: cargo is " + Round(cargo_weight_kg, 2) + " kg
                          "drone maximum is " + Round(drone_max_payload_kg, 2) + " kg."
6:     Return False, rejection_reason
7: end if

8: Return True, "Payload within safe manufacturer envelope."
```

---

7. Algorithm 6: Priority Best-Fit Multi-Drone Allocation

7.1 Concept & Intuition

When a disaster strikes or multiple hospitals submit simultaneous requests, available drones are scarce. Naive assignment can result in a routine supply order claiming the only available heavy-lift drone, leaving a critical blood request stranded. The **Priority Best-Fit Algorithm** solves this bipartite matching problem in two stages: 1. **Clinical Triage Ranking:** Sort requests by priority weight: $\text{CRITICAL}\ (4) > \text{HIGH}\ (3) > \text{MEDIUM}\ (2) > \text{LOW}\ (1)$. 2. **Best-Fit Capacity Matching:** For each high-priority request, find all eligible available drones that can physically carry the payload and reach the destination. Select the drone that **minimizes wasted payload headroom** ($\Delta m = m_{\text{max}} - m_{\text{cargo}}$).

7.2 Formal Pseudocode

```
Algorithm: PriorityBestFitAllocation
Input: pending_requests (List of DeliveryRequest), available_drones (List of Drone), weather
Output: MultiDroneAllocationPlan (matched_assignments, unmatched_requests)

1: priority_weights = {"CRITICAL": 4, "HIGH": 3, "MEDIUM": 2, "LOW": 1}
2: sorted_requests = Sort(pending_requests, by=[
       -priority_weights[req.priority],
       req.created_at ASC,
       req.id ASC
   ])

3: remaining_drones = Copy(available_drones)
4: matches = []
5: unmatched = []

6: for req in sorted_requests do:
7:     eligible_drone_candidates = []
8:     for drone in remaining_drones do:
9:         if drone.max_payload_kg >= req.payload_weight_kg then:
10:            opt_res = RouteOptimizer.optimize(req, drone, weather)
11:            if opt_res.is_feasible == True then:
12:                headroom_kg = drone.max_payload_kg - req.payload_weight_kg
13:                eligible_drone_candidates.append({
                       "drone": drone,
                       "route": opt_res.selected_route,
                       "headroom_kg": headroom_kg,
                       "energy_wh": opt_res.selected_route.energy_wh
                   })
14:            end if
15:        end if
16:    end for

17:    if eligible_drone_candidates is NOT EMPTY then:
           // Best-Fit: minimize wasted payload capacity headroom, break ties by energy
18:        best_match = Sort(eligible_drone_candidates, by=[cand.headroom_kg ASC, cand.energy_wh A
19:        matches.append(CreateMatch(req, best_match.drone, best_match.route))
20:        remaining_drones.remove(best_match.drone)  // Atomic claim
21:    else:
22:        reason = "NO_AVAILABLE_DRONES" if remaining_drones is EMPTY else "INSUFFICIENT_DRONE_CA
23:        unmatched.append(CreateUnmatched(req, reason))
24:    end if
25: end for

26: Return MultiDroneAllocationPlan(matches, unmatched, idle_drones=remaining_drones)
```

---

8. Algorithm 7: First-Come-First-Served (FCFS) Baseline Allocation

8.1 Concept & Intuition

The FCFS algorithm represents the common default operational strategy used in standard queuing systems. Requests are processed strictly in arrival order ($\text{created_at ascending}$), and each request is assigned the first available drone capable of completing the mission, completely ignoring priority levels.

8.2 Formal Pseudocode

```
Algorithm: FCFSGreedyAllocation
Input: pending_requests (List of DeliveryRequest), available_drones (List of Drone), weather
Output: MultiDroneAllocationPlan

1: sorted_requests = Sort(pending_requests, by=[req.created_at ASC, req.id ASC])
2: remaining_drones = Copy(available_drones)
3: matches = []
4: unmatched = []

5: for req in sorted_requests do:
6:     assigned = False
7:     for drone in remaining_drones do:
8:         if drone.max_payload_kg >= req.payload_weight_kg then:
9:             opt_res = RouteOptimizer.optimize(req, drone, weather)
10:            if opt_res.is_feasible == True then:
11:                matches.append(CreateMatch(req, drone, opt_res.selected_route))
12:                remaining_drones.remove(drone)
13:                assigned = True
14:                break  // Greedy assignment: take first matching drone
15:            end if
16:        end if
17:    end for
18:    if not assigned then:
19:        unmatched.append(CreateUnmatched(req, "NO_AVAILABLE_DRONES"))
20:    end if
21: end for

22: Return MultiDroneAllocationPlan(matches, unmatched, idle_drones=remaining_drones)
```

---

9. Algorithm 8: Dynamic In-Flight Rerouting Engine

9.1 Concept & Intuition

While a drone is in transit, sudden environmental hazards (e.g. localized thunderstorms, crosswind gusts) can render the pre-planned route hazardous. The dynamic rerouting engine evaluates replacement routes starting from the drone's instantaneous mid-air coordinate $(\phi_{\text{curr}}, \lambda_{\text{curr}})$ to the destination, switching routes safely under the `SAFETY_MANDATED` policy and logging an immutable audit record.

9.2 Formal Pseudocode

```
Algorithm: DynamicInFlightReroute
Input: simulation_id, updated_weather, policy_name="SAFETY_MANDATED", reason
Output: DynamicRerouteResult(switched, original_route_id, new_route_id, explanation)

1: sim = db.get(SimulationRun, simulation_id)
2: if sim.status not in ["RUNNING", "PAUSED"] then:
3:     raise InvalidStateError("Dynamic rerouting only valid for active flights.")
4: end if

5: curr_lat = sim.current_latitude; curr_lon = sim.current_longitude
6: dest_lat = sim.assignment.request.destination_latitude
7: dest_lon = sim.assignment.request.destination_longitude

8: remaining_dist = Haversine(curr_lat, curr_lon, dest_lat, dest_lon)
9: if remaining_dist < 0.10 then: // Within 100 meters of landing pad
10:    Return DynamicRerouteResult(switched=False, explanation="Drone on final approach. Reroutin
11: end if

12: opt_res = RouteOptimizer.optimize(curr_lat, curr_lon, dest_lat, dest_lon, sim.drone, updated_
13: if opt_res.feasible_count == 0 then:
14:    Return DynamicRerouteResult(switched=False, explanation="No replacement candidate safe und
15: end if

16: best_cand = opt_res.selected_route
17: should_switch = False

18: if policy_name == "SAFETY_MANDATED" then:
19:    should_switch = True
20: else if policy_name == "OBJECTIVE_IMPROVEMENT" then:
21:    if (sim.remaining_energy_wh - best_cand.energy_wh) / sim.remaining_energy_wh >= 0.05 then:
22:        should_switch = True
23:    end if
24: end if

25: if should_switch == True then:
26:    new_route = CreateRouteRecord(sim.request_id, algorithm="REROUTE:" + best_cand.id, best_ca
27:    db.add(new_route); db.flush()
28:    AppendWaypoints(new_route.id, best_cand.waypoints)
29:    sim.assignment.route_id = new_route.id  // Atomic point to new route
30:    AppendAuditLog(sim.parameters_json["reroute_history"], orig_route_id=sim.route_id, new_rou
31:    db.commit()
32:    Return DynamicRerouteResult(switched=True, new_route_id=new_route.id)
33: else:
34:    Return DynamicRerouteResult(switched=False, explanation="Switch policy threshold not met."
35: end if
```

---

10. Algorithm 9: Advisory Ridge Regression Prediction

10.1 Concept & Intuition

To provide dispatchers with rapid duration and energy forecasts without running iterative numerical flight simulations, an advisory Machine Learning module is trained using L2-regularized Ridge Regression. - Model minimizes squared error while penalizing large weights to prevent overfitting:

$$\min_{w} \frac{1}{2N} \sum_{i=1}^{N} (y_i - x_i^T w)^2 + \frac{\alpha}{2} \|w\|_2^2$$

- Closed-form analytical solution:

$$w^* = (X^T X + \alpha I)^{-1} X^T y$$

10.2 Feature Vector ($D = 6$)

$$x = \begin{bmatrix} d \\ m \\ v_w \\ v_c \\ m \cdot d \\ v_w^2 \end{bmatrix} = \begin{bmatrix} \text{Distance (km)} \\ \text{Cargo Mass (kg)} \\ \text{Wind Speed (m/s)} \\ \text{Cruise Speed (km/h)} \\ \text{Payload } \times \text{ Distance Interaction} \\ \text{Aerodynamic Drag Velocity Squared} \end{bmatrix}$$

10.3 Formal Pseudocode

```
Algorithm: RidgeRegressionTrainingAndInference
Input: Training Matrix X_train (N x 6), Labels y_train (N), Test Sample x_new (6), alpha=1.0
Output: y_pred (float)

// 1. Training Phase (Closed-Form Analytical Inversion)
1: N, D = Dimensions(X_train)
2: I_reg = IdentityMatrix(D)
3: I_reg[0, 0] = 0.0  // Do not regularize intercept bias column

4: XtX = Transpose(X_train) * X_train
5: Xty = Transpose(X_train) * y_train
6: weights = MatrixInverse(XtX + alpha * I_reg) * Xty

// 2. Inference Phase
7: y_pred = DotProduct(x_new, weights)
8: Return Max(0.0, y_pred)  // Physical constraint: duration and energy cannot be negative
```